

Atividade Prática: Fila de Prioridade Hospitalar (Protocolo de Manchester)

Disciplina: Estruturas de Dados e Algoritmos

Nível: Mestrado em Ciência da Computação

1. Contextualização e Objetivo

Nos sistemas de saúde modernos, o atendimento por ordem de chegada (FIFO) é frequentemente ineficiente e perigoso. Para mitigar isso, hospitais utilizam protocolos de triagem para classificar o risco de cada paciente no momento da chegada. O mais famoso deles é o **Protocolo de Manchester**, que divide os pacientes em 5 níveis de gravidade associados a cores:

Gravidade	Cor	Prioridade (Valor Numérico)	Tempo Limite Alvo
Emergência	Vermelho	5	Imediato
Muito Urgente	Laranja	4	10 minutos
Urgente	Amarelo	3	120 minutos
Pouco Urgente	Verde	2	240 minutos
Não Urgente	Azul	1	240 minutos

Seu objetivo nesta atividade:

Implementar um sistema interativo de simulação de triagem e atendimento médico em C. O sistema deverá gerenciar a fila de pacientes utilizando o Tipo Abstrato de Dados (TAD) **Heap Máximo** desenvolvido em aula, permitindo comparar na prática o impacto de duas abordagens de implementação física: **Estática (Vetor)** e **Dinâmica (Ponteiros/Nós)**.

2. Requisitos de Implementação

Você deverá construir um programa em C (main.c) que use a especificação do TAD fornecida no arquivo heap.h.

A. Estrutura do Paciente

Como o nosso Heap armazena o tipo de dado int, a sua simulação deverá mapear as prioridades de Manchester para números inteiros de 1 a 5. Para manter um registro do paciente associado ao seu valor no Heap, você pode adotar duas estratégias (escolha **uma** delas para implementar):

- **Abordagem Simplificada (Recomendada para nivelamento):** O Heap armazena apenas prioridades (int). Você mantém uma estrutura paralela ou simplesmente gera nomes genéricos ao extrair (ex: ao remover um valor 5, exibe "*Chamando paciente de Emergência (Prioridade 5)*").

- **Abordagem Avançada (Desafio):** Altere levemente os arquivos de implementação para que o Heap armazene uma struct Paciente em vez de um inteiro int. A struct deve conter:

```
typedef struct {
    char nome[50];
    int prioridade;
    size_t ordem_chegada; // Opcional: para desempate de mesma
    prioridade (FIFO)
} Paciente;
```

B. O Fluxo de Simulação (Interface Baseada em Caracteres)

O programa principal deve ler comandos do usuário via console (entrada padrão) para simular o fluxo do pronto-socorro em tempo real. Os comandos válidos são:

- **I <nome> <prioridade>:** Registra a chegada de um paciente (insere no Heap).
 - *Exemplo:* I Carlos 4 (Insere Carlos com prioridade Laranja/Muito urgente).
- **C:** Chama o próximo paciente para o consultório médico (remove o elemento de maior prioridade do Heap e o exibe na tela).
 - *Exemplo:* Próximo paciente: Carlos (Prioridade 4) dirigido ao Consultório 1.
- **T:** Mostra o topo atual (exibe o nível de prioridade do caso mais urgente em espera, sem removê-lo).
- **M:** Mostra o estado atual da estrutura de dados por níveis (invocando a função heap_mostrar).
- **S:** Sai do programa e libera toda a memória alocada (heap_destruir).

3. Roteiro de Testes Recomendado

Para validar a corretude das operações de subida (sift-up) e descida (sift-down), execute a seguinte sequência de ações no seu simulador e use o comando M (mostrar) para desenhar a árvore mentalmente após cada etapa:

1. **Chegada consecutiva de pacientes leves:**
 - Inserir prioridade 1 (Azul).
 - Inserir prioridade 2 (Verde).
 - *Espera-se:* O 2 deve subir para a raiz.
2. **Chegada de uma emergência crítica:**
 - Inserir prioridade 5 (Vermelho).
 - *Espera-se:* O 5 deve propagar até o topo do Heap.
3. **Primeiro atendimento:**
 - Executar o comando de Chamada (C).
 - *Espera-se:* A emergência (5) deve ser removida, e a propriedade do heap deve ser

reajustada descendo o substituto temporário.

4. Reflexão Conceitual (Relatório de Pós-Graduação)

Como alunos de mestrado, a atividade prática não se encerra na execução do código. Você deve anexar um breve parágrafo (de 10 a 15 linhas) respondendo de forma crítica:

Questão: Execute a sua simulação compilando o seu `main.c` primeiro com o `heap_vetor.c` e, em seguida, com o `heap_ponteiro.c`. Do ponto de vista de eficiência de hardware moderno (cache, paginação) e uso de memória, quais são os prós e contras práticos de cada arquitetura na escala de um hospital municipal (ex: 2.000 atendimentos diários)? Por que a notação assintótica $O(\log N)$ falha em nos dar esse panorama completo?

5. Critérios de Avaliação

1. **Corretude do Fluxo:** O paciente de maior prioridade deve sempre ser o primeiro a ser chamado, independentemente da ordem em que entrou.
2. **Gerenciamento de Memória:** O código deve compilar sem vazamentos de memória (*memory leaks*). Use o `valgrind` para certificar-se de que o comando `S` (`Sair`) libera todos os nós e dados alocados.
3. **Qualidade de Código:** Uso correto de tipos (`size_t` para índices/tamanhos), modularização e comentários explicativos.

(heap.h)

```
#ifndef HEAP_H
#define HEAP_H

#include <stddef.h>
typedef struct heap Heap;

/* Cria um heap com capacidade inicial */
Heap *heap_criar(size_t capacidade);

/* Insere um elemento no heap (heap máximo) */
void heap_inserir(Heap *h, int valor);

/* Remove e retorna o maior elemento */
int heap_remove(Heap *h);

/* Retorna o maior elemento sem remover */
int heap_topo(const Heap *h);

/* Retorna o número de elementos */
size_t heap_tamanho(const Heap *h);

/* Mostra o heap por níveis */
void heap_mostrar(const Heap *h);

/* Libera memória */
void heap_destruir(Heap *h);

#endif
```

heap.c

```
#include "heap.h"
#include <stdlib.h>
#include <stdio.h>

struct heap {
    int *dados;
    size_t tamanho;
    size_t capacidade;
};

/* ----- Funções auxiliares
----- */

static void troca(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

/* Heap máximo: o maior sobe */
static void subir(Heap *h, size_t i) {
    while (i > 0) {
        size_t pai = (i - 1) / 2;

        if (h->dados[i] <= h->dados[pai])
            return;

        troca(&h->dados[i], &h->dados[pai]);
        i = pai;
    }
}
```

```

}

/* Heap máximo: o maior desce */
static void descer(Heap *h, size_t i) {
    size_t tamanho = h->tamanho;

    /* Enquanto existir filho esquerdo */
    while (2*i + 1 < tamanho) {
        size_t esq = 2*i + 1;
        size_t dir = 2*i + 2;
        size_t maior = esq;

        if (dir < tamanho && h->dados[dir] > h->dados[esq])
            maior = dir;

        if (h->dados[i] >= h->dados[maior])
            return;

        troca(&h->dados[i], &h->dados[maior]);
        i = maior;
    }
}

/* ----- Interface pública
----- */

Heap *heap_criar(size_t capacidade) {
    Heap *h = malloc(sizeof(Heap));
    h->dados = malloc(sizeof(int) * capacidade);
    h->tamanho = 0;
    h->capacidade = capacidade;
}

```

```

        return h;
    }

void heap_inserir(Heap *h, int valor) {
    if (h->tamanho == h->capacidade) {
        h->capacidade *= 2;
        h->dados = realloc(h->dados, sizeof(int) *
h->capacidade);
    }

    h->dados[h->tamanho] = valor;
    subir(h, h->tamanho);
    h->tamanho++;
}

int heap_remover(Heap *h) {
    if (h->tamanho == 0) {
        fprintf(stderr, "Erro: heap vazio\n");
        exit(1);
    }

    int raiz = h->dados[0];
    h->tamanho--;
    h->dados[0] = h->dados[h->tamanho];
    descer(h, 0);

    return raiz;
}

int heap_topo(const Heap *h) {
    if (h->tamanho == 0) {

```

```

        fprintf(stderr, "Erro: heap vazio\n");
        exit(1);
    }
    return h->dados[0];
}

size_t heap_tamanho(const Heap *h) {
    return h->tamanho;
}

/* ----- Mostrar heap por níveis
----- */

void heap_mostrar(const Heap *h) {
    if (h->tamanho == 0) {
        printf("(heap vazio)\n");
        return;
    }

    size_t nivel = 0;
    size_t elementos_no_nivel = 1;
    size_t i = 0;

    while (i < h->tamanho) {
        printf("Nível %zu: ", nivel);

        for (size_t j = 0; j < elementos_no_nivel && i <
h->tamanho; j++, i++) {
            printf("%d ", h->dados[i]);
        }
    }
}

```

```
        printf("\n");  
        nivel++;  
        elementos_no_nivel *= 2;  
    }  
}
```

```
void heap_destruir(Heap *h) {  
    free(h->dados);  
    free(h);  
}
```